

System Design Assignment

Problem Statement :

Build a small movie ticket booking system for a single cinema (like PVR or INOX), as a menu-driven C++ console program.

A customer should be able to see which movies are playing, pick a show, see which seats are free, book seats, pay, get a ticket, and cancel a booking.

A.Requirement Analysis

- Functional requirements describe the specific functions or actions that the system must perform.
- Non-functional requirements describe how well the system should work or the qualities/constraints the system should have, rather than a specific function.

Code :

```
main.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      cout << "=====\n";
7      cout << "      MOVIE TICKET BOOKING SYSTEM\n";
8      cout << "=====\n";
9
10     cout << "\nProject: System Design Assignment\n";
11     cout << "Day 1: Requirement Analysis\n";
12
13     cout << "\n--- Functional Requirements ---\n";
14     cout << "1. List Movies\n";    //display currently available movies
15     cout << "2. List Shows\n";    //display currently available shows for a selected movie with screen no. and start time
16     cout << "3. Display Seat\n";    //display all seats with their status (available or booked), seat no. and type
17     cout << "4. Book Seats\n";    //customer can book available seats
18     cout << "5. Price Calculation\n"; //silver : 150, gold :250, platinum:400
19     cout << "6. Make Payment\n";    //customer can make payment for booked seats
20     cout << "7. Print Ticket\n";    //ticket display booking ID, movie, screen, time, amount, seat no. and type
21     cout << "8. Cancel Booking\n"; //booking can be cancelled and seats are released
22
23     cout << "\n--- Non-Functional Requirements ---\n";
24     cout << "1. Modularity\n";    //separate class for separate responsibilities
25     cout << "2. Extensibility\n";    //new features can be added without affecting existing code
26     cout << "3. Input Validation\n"; //validate user inputs
27     cout << "4. Maintainability\n"; //code should be easy to maintain and update
28
29     cout << "\n=====\n";
30     cout << "      Requirement Analysis Completed!\n";
31     cout << "=====\n";
32
33     return 0;
34 }
```

Output :

```
riya@Riyas-MacBook-Air-2 System_Design_Assign_1 % g++ main.cpp
riya@Riyas-MacBook-Air-2 System_Design_Assign_1 % ./a.out
=====
                MOVIE TICKET BOOKING SYSTEM
=====

Project: System Design Assignment
Day 1: Requirement Analysis

--- Functional Requirements ---
1. List Movies
2. List Shows
3. Display Seat
4. Book Seats
5. Price Calculation
6. Make Payment
7. Print Ticket
8. Cancel Booking

--- Non-Functional Requirements ---
1. Modularity
2. Extensibility
3. Input Validation
4. Maintainability

=====
                Requirement Analysis Completed!
=====
```

B. Noun Verb Analysis table

- Noun-Verb Analysis is a technique used in object-oriented system design to identify the classes and their responsibilities from a problem statement.

STEP B — NOUN-VERB ANALYSIS

Noun & Verb Found	Keep as a Class?	Reason
Movie – display, store	Yes	Has its own data and identity (title, language, duration).
Seat – book, release	Yes	Has number and type, represents a physical seat.
Seat Layout – display	No	A view of a show's seats, not a separate thing → use a print/display method.
Cinema – contain, manage	Yes	Represents the theatre and owns its screens.
Screen – contain, display	Yes	Represents an auditorium and owns seats.
Show – schedule, display	Yes	Represents a movie on a screen at a specific time.
ShowSeat – book, release	Yes	Maintains the status of one seat for one show.
Customer – book, cancel	Yes	Has customer information and identity.
Booking – create, cancel	Yes	Stores booking details, seats, amount and status.
Payment – pay	Yes	Defines the payment contract.
Price – calculate	No	A value/calculation, not an independent entity → use PriceCalculator.
Ticket – print	No	Printing is a behaviour → handled by TicketPrinter.
Booking Flow – run	No	A process, not an entity → handled by BookingService.

- Core entity classes are classes that represent the important real-world things or objects in the system and mainly store their data and state.

STEP C — CORE ENTITY CLASSES

Class	Data Members	Access Modifiers	Responsibility / Methods	Must NOT Do
Movie	title, language, duration	private	Store movie details (getTitle(), getLanguage(), getDuration())	Must NOT handle booking or payment.
Seat	number, type	private	Represent one physical seat (getNumber(), getType())	Must NOT store booking status.
Screen	screenNumber, seats	private	Own and manage seats (getScreenNumber(), getSeats())	Must NOT handle payment.
Cinema	name, screens	private	Own and manage screens (getName(), getScreens())	Must NOT calculate booking prices.
Show	movie, screen, time, showSeats	private	Represent one movie screening (getMovie(), getScreen(), getTime())	Must NOT process payment.
ShowSeat	seat, status	private	Track seat status for one show (isAvailable(), bookSeat(), releaseSeat())	Must NOT change physical seat number/type.
Customer	name, phone	private	Store customer information (getName(), getPhone())	Must NOT manage payment.
Booking	bookingId, show, seats, totalAmount, status	private	Store booking details (getBookingId(), getSeats(), getTotalAmount())	Must NOT print ticket or process payment.

- Behaviour or service classes are classes whose main purpose is to perform operations or business logic rather than represent a physical/domain entity.

STEP C — BEHAVIOUR / SERVICE CLASSES

Class	Data Members	Access Modifiers	Responsibility / Methods	Must NOT Do
Payment (abstract)	–	public (methods)	Define payment contract (+ pay(double amount))	Must NOT implement a specific payment method.
UpiPayment	upiId, phoneNumber	private	Process UPI payment (+ pay(double amount))	Must NOT manage bookings.
CardPayment	cardNumber, cardHolderName, expiryDate	private	Process Card payment (+ pay(double amount))	Must NOT manage bookings.
CashPayment	–	private	Process Cash payment (+ pay(double amount))	Must NOT manage bookings.
PriceCalculator	silverPrice, goldPrice, platinumPrice	private	Calculate total amount for selected seats (+ calculateTotal(seats))	Must NOT book seats or process payment.
TicketPrinter	–	public	Format and print ticket (+ printTicket(booking))	Must NOT modify booking data.
BookingService	cinema, priceCalculator, ticketPrinter	private	Run the booking flow end-to-end (+ bookSeats(), cancelBooking())	Must NOT implement payment logic directly.
Cinema / Main Menu	bookingService	private	Display menu and read user input (+ showMenu(), handleInput())	Must NOT contain business logic.

C. Relationship table

- A relationship in UML shows how two classes are connected or interact with each other in a system.
- Composition is a strong "has-a" relationship where the child object depends on the parent for its lifetime.
- Aggregation is a weaker "has-a" relationship where one class uses or contains another object, but the contained object can exist independently.
- Association is a general relationship where two classes interact or communicate with each other, but neither class owns the other.
- Inheritance is a relationship in which one class is a specialized version of another class and inherits its properties and behaviour.

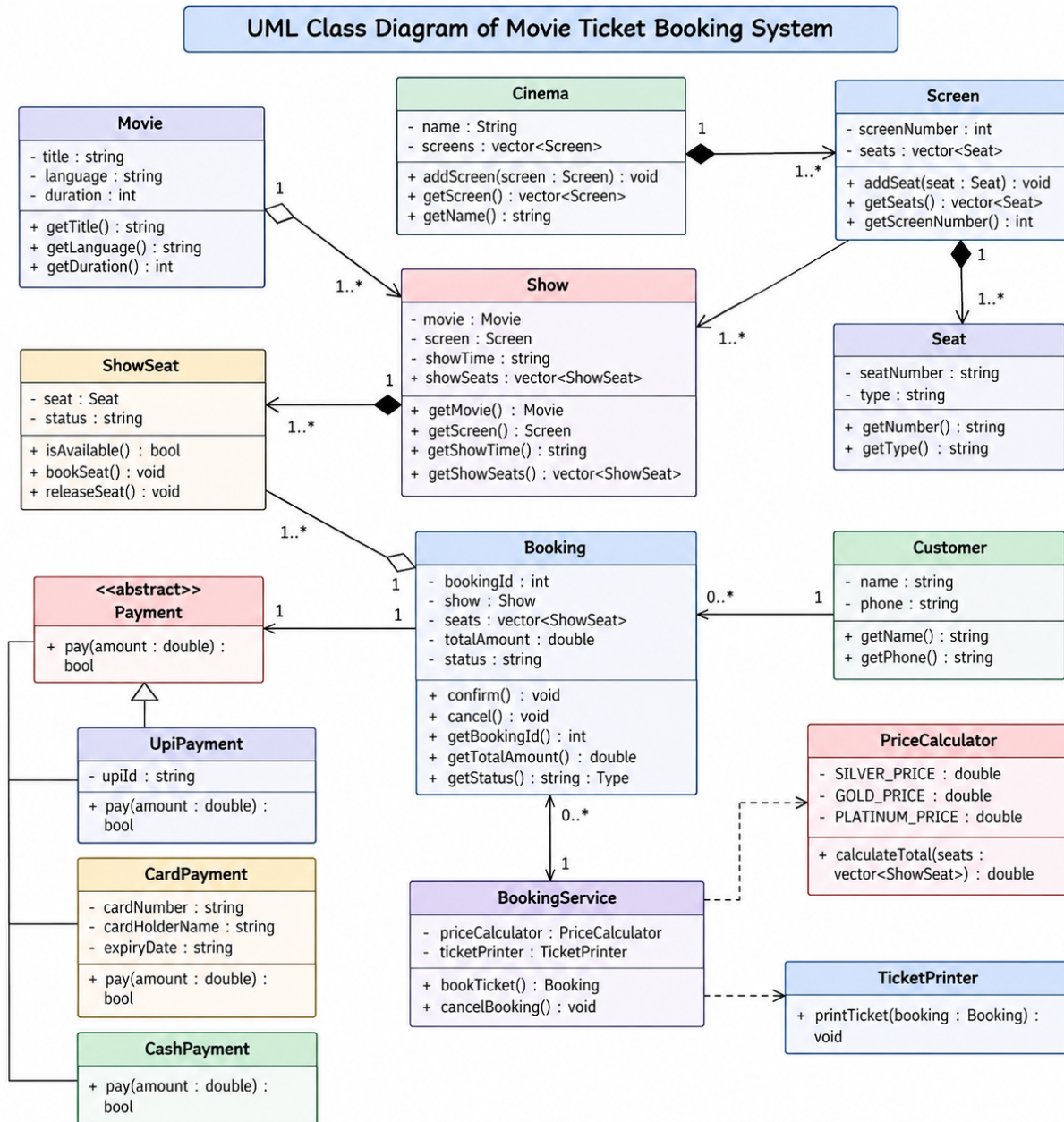
STEP D — COMPLETE RELATIONSHIP TABLE

Every choice is justified using the lifetime test.

Pair	Choice	Lifetime-test justification	Multiplicities
Cinema – Screen	Composition	Cinema creates and owns its Screens. If Cinema is destroyed, its Screens do not exist.	1 .. *
Screen – Seat	Composition	Screen creates and owns its Seats. If Screen is destroyed, its Seats do not exist.	1 .. *
Show – Movie	Aggregation	Show uses an existing Movie. If a Show is destroyed, the Movie still exists.	1 .. 1
Show – Screen	Aggregation	Show uses an existing Screen. If a Show is destroyed, the Screen still exists.	1 .. 1
Show – ShowSeat	Composition	Show creates and owns its ShowSeats. If a Show is destroyed, its ShowSeats do not exist.	1 .. *
Booking – Customer	Association	Booking uses a Customer. If a Booking is destroyed, the Customer still exists.	* .. 1
Booking – ShowSeat	Aggregation	Booking refers to existing ShowSeats. If a Booking is destroyed, the ShowSeats still exist.	1 .. *
Booking – Payment	Association	Booking uses a Payment. If a Booking is destroyed, the Payment object still exists.	1 .. 1
Payment – UpiPayment	Inheritance	UpiPayment IS-A Payment (specialized type).	—
Payment – CardPayment	Inheritance	CardPayment IS-A Payment (specialized type).	—
Payment – CashPayment	Inheritance	CashPayment IS-A Payment (specialized type).	—
BookingService – Booking	Association	BookingService manages Bookings, but a Booking can exist independently (e.g., stored).	1 .. *

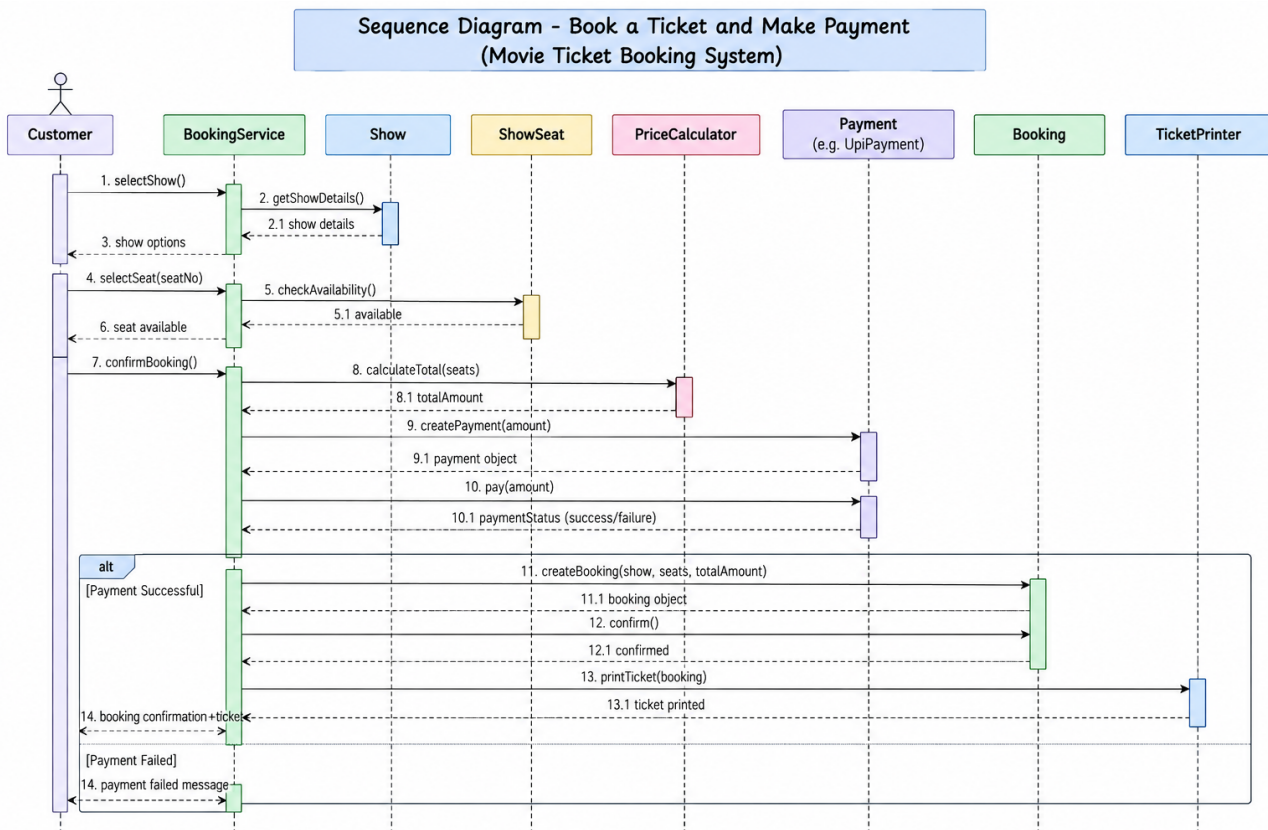
D. UML Diagram

A UML diagram is a standardized visual representation used to design, understand, and communicate the structure and behaviour of a software system.



E. Sequence Diagram

A sequence diagram is a UML diagram that shows the sequence of interactions or messages between different objects/classes over time to complete a particular task or use case.



F. Modular working code and Demo Output

Github Repository link (Code) : <https://github.com/RiyaBisht1708/Movie-Ticket-Booking-System>

1. All movies are displayed.

```
● riya@Riyas-MacBook-Air-2 Movie_Ticket_Booking_System % g++ main.cpp
○ riya@Riyas-MacBook-Air-2 Movie_Ticket_Booking_System % ./a.out

=====
MOVIE TICKET BOOKING SYSTEM
=====

1. List Movies
2. Display Shows and Seats
3. Book Ticket and Make Payment
4. Cancel Booking
5. Display All Bookings
0. Exit
Enter choice: 1

--- Movies ---
1. 3 Idiots | Hindi | 170 min
2. Inception | English | 148 min
3. Dangal | Hindi | 161 min
```

2. Ticket booking done.

```
riya@Riyas-MacBook-Air-2 Movie_Ticket_Booking_System % ./a.out

=====
MOVIE TICKET BOOKING SYSTEM
=====

1. List Movies
2. Display Shows and Seats
3. Book Ticket and Make Payment
4. Cancel Booking
5. Display All Bookings
0. Exit
Enter choice: 3

--- Shows ---
1. Screen 1 | 06:00 PM | 3 Idiots
2. Screen 1 | 09:00 PM | 3 Idiots
3. Screen 2 | 07:00 PM | Inception
4. Screen 2 | 10:00 PM | Dangal
Select show: 4

Seat Layout:
A1 - SILVER [AVAILABLE]
A2 - SILVER [AVAILABLE]
B1 - GOLD [AVAILABLE]
B2 - GOLD [AVAILABLE]
C1 - PLATINUM [AVAILABLE]
C2 - PLATINUM [AVAILABLE]
How many seats? 1
Enter seat 1: B1
1. UPI
2. Card
3. Cash
Choose payment: 3

Customer: Riya
Processing Cash payment of Rs.250...
Payment successful. Booking confirmed.
```

3. Booked Seat is shown.

```
=====
MOVIE TICKET
=====
Booking ID : BK1001
Movie      : Dangal
Screen     : 2
Show Time  : 10:00 PM
Seats      : B1
Total      : Rs.250.00
Status     : CONFIRMED
=====

Seat Layout:
A1 - SILVER [AVAILABLE]
A2 - SILVER [AVAILABLE]
B1 - GOLD [BOOKED]
B2 - GOLD [AVAILABLE]
C1 - PLATINUM [AVAILABLE]
C2 - PLATINUM [AVAILABLE]
```

4. Edge case : Trying to book an already booked seat, No crash only error message displayed.

```
=====
MOVIE TICKET BOOKING SYSTEM
=====
1. List Movies
2. Display Shows and Seats
3. Book Ticket and Make Payment
4. Cancel Booking
5. Display All Bookings
0. Exit
Enter choice: 3

--- Shows ---
1. Screen 1 | 06:00 PM | 3 Idiots
2. Screen 1 | 09:00 PM | 3 Idiots
3. Screen 2 | 07:00 PM | Inception
4. Screen 2 | 10:00 PM | Dangal
Select show: 4

Seat Layout:
A1 - SILVER [AVAILABLE]
A2 - SILVER [AVAILABLE]
B1 - GOLD [BOOKED]
B2 - GOLD [AVAILABLE]
C1 - PLATINUM [AVAILABLE]
C2 - PLATINUM [AVAILABLE]
How many seats? 1
Enter seat 1: B1
1. UPI
2. Card
3. Cash
Choose payment: 3

Customer: Riya
Seat B1 is already BOOKED. Booking rejected.
```


5. Cancel booking.

```
=====
MOVIE TICKET BOOKING SYSTEM
=====
1. List Movies
2. Display Shows and Seats
3. Book Ticket and Make Payment
4. Cancel Booking
5. Display All Bookings
0. Exit
Enter choice: 4

--- Bookings ---
1. Booking BK1001 | Dangal | 10:00 PM | CONFIRMED
2. Booking BK1002 | Dangal | 10:00 PM | PENDING
Select booking to cancel: 1
Booking BK1001 cancelled. Seats are AVAILABLE again.
```

6. Edge case : Booking an invalid seat number.

```
=====
MOVIE TICKET BOOKING SYSTEM
=====
1. List Movies
2. Display Shows and Seats
3. Book Ticket and Make Payment
4. Cancel Booking
5. Display All Bookings
0. Exit
Enter choice3

--- Shows ---
1. Screen 1 | 06:00 PM | 3 Idiots
2. Screen 1 | 09:00 PM | 3 Idiots
3. Screen 2 | 07:00 PM | Inception
4. Screen 2 | 10:00 PM | Dangal
Select show: 4

Seat Layout:
A1 - SILVER [AVAILABLE]
A2 - SILVER [AVAILABLE]
B1 - GOLD [AVAILABLE]
B2 - GOLD [AVAILABLE]
C1 - PLATINUM [AVAILABLE]
C2 - PLATINUM [AVAILABLE]
How many seats? D1
Invalid number of seats.

=====
MOVIE TICKET BOOKING SYSTEM
=====
1. List Movies
2. Display Shows and Seats
3. Book Ticket and Make Payment
4. Cancel Booking
5. Display All Bookings
0. Exit
Enter choice: Invalid input.
```

G. SOLID Mapping

- **S — Single Responsibility Principle:**

Booking, PriceCalculator, TicketPrinter, and BookingService have separate responsibilities.

- **O — Open/Closed Principle:**

New payment methods can be added by extending Payment without modifying BookingService.

- **L — Liskov Substitution Principle:**

UpiPayment, CardPayment, and CashPayment can be used through the Payment abstraction.

- **I — Interface Segregation Principle:**

Payment exposes only the common pay() operation, so payment classes are not forced to implement unrelated operations.

- **D — Dependency Inversion Principle:**

BookingService receives a Payment& rather than directly creating a concrete payment implementation

SOLID Mapping for Movie Ticket Booking System			
Principle	Meaning (in brief)	Where applied (Classes)	How it is applied in our system
Single Responsibility Principle (SRP)	A class should have one reason to change (i.e., one responsibility).	Booking, PriceCalculator, TicketPrinter, BookingService	• Booking only stores booking details. • PriceCalculator only calculates total amount. • TicketPrinter only prints the ticket. • BookingService only coordinates the booking flow.
Open/Closed Principle (OCP)	Software entities should be open for extension but closed for modification.	Payment and its subclasses (UpiPayment, CardPayment, CashPayment)	• New payment methods (e.g., NetBankingPayment) can be added by creating a new class derived from Payment. • No changes are required in existing classes like BookingService.
Liskov Substitution Principle (LSP)	Objects of a child class should be usable wherever the parent class is expected without breaking the system.	Payment, UpiPayment, CardPayment, CashPayment	• Every payment class can be used through a Payment* or Payment& reference. • Each subclass implements pay(amount) and works in the same way, without requiring extra setup.
Interface Segregation Principle (ISP)	A class should not be forced to depend on methods that it does not need.	Payment	• The Payment interface contains only the common operation pay(amount). • We did not force all payment classes to implement unrelated methods like refund(), which may not be applicable to all.
Dependency Inversion Principle (DIP)	High-level classes should depend on abstractions rather than concrete implementations.	BookingService → Payment	• BookingService receives a Payment reference instead of creating a specific payment object (e.g., CardPayment) itself. • This reduces coupling and allows easy addition of new payment types.